SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 10%

Duration :60 Mins
On Class
QUESTIONS: 50 **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I SHEIK MOHAMED_192511145(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SHEIK MOHAMED M

Name : Sheik mohamed

Reg NO : 192511145

course : compiler design

course code : CSA1408

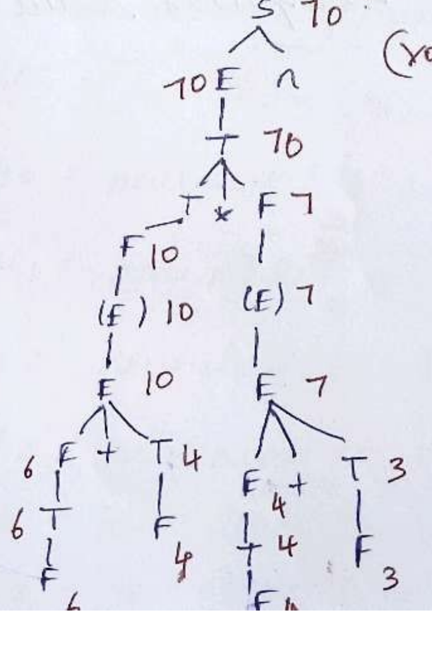
CO-3 - Assessment tool-3
Sanario Based

Q1 Syntax - Directed Definition (SDD):

* $S \rightarrow En$ { S.val = E.val }
* $E \rightarrow E_1 + T$ { E.val = E_1.val + T.val }
* $E \rightarrow E_1 - T$ { E.val = E_1.val - T.val }
* $E \rightarrow T$ { E.val = T.val }
* $T \rightarrow T_1 * F$ { T.val = T_1.val * F.val }
* $T \rightarrow T_1 / F$ { T.val = F.val }
* $T \rightarrow F$ { T.val = F.val }
* $F \rightarrow (E)$ { F.val = E.val }
* $F \rightarrow$ digit { F.val = digit.lexval }

Annotated Parse tree for (6+4) * (4+3)

(val = 70)



Left subtree : (6+4)

F → digit (6) => F.val = 6

T → F => T.val = 6

E → T => E.val = 6

F → digit (4) => F.val = 4

T → F => T.val = 4

E → E(6) + T(4) = E.val (10)

F → (E(10)) => F.val = 10

T → F => T.val = 10

Right subtree (4+3)

F → digit(4) => F.val = 4

T → F => T.val = 4

E → T => E.val = 4

F → digit (3) => F.val = 3

T → F => T.val = 3

E → E(4) + T(3) = E.val = 7

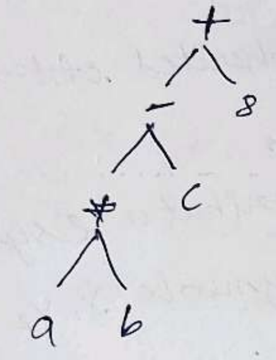
F → (E (7)) => F.val = 7

Final computed value : 70

Syntax tree for:

$a * b - c + 8$

considering standard operator precedence ($* > - = +$) and left to right associativity.



syntax tree

Syntax - Directed definition (SDD

* $E \rightarrow E_1 + T$ { E.node = new node('+', E_1.node, T.node) }
* $E \rightarrow E_1 - T$ { E.node = new Node('-', E_1.Node, T.Node }
* $E \rightarrow T$ { E.Node = T.node }
* $T \rightarrow T_1 * F$ { T.Node = new Node('*', T_1.Node, F.Node }
* $T \rightarrow F$ { T.Node = F.Node }
* $F \rightarrow id$ { F.Node = new Leaf (id, id.entry) }
* $F \rightarrow num$ { F.Node = new leaf (num, num.val }

Attribute Evaluation Sequence:

1) $Leaf_1$ = new leaf (id, a)
2) $Leaf_2$ = new leaf (id, b)
3) $Node_1$ = new node ('*', $Leaf_1$, $Leaf_2$)
4) $Leaf_3$ = new leaf (id, c)
5) $Node_2$ = new node ('-', $Node_1$, $Leaf_3$)
6) $Leaf_4$ = new leaf (num, 8)
7) $Node_3$ = new Node ('+', $Node_2$, $Leaf_4$)

Q3 L—Attributed Definition conditions

An SDD is L-attributed if each inherited attribute of a symbol on the right hand side of a production $A \rightarrow Y_1 Y_2 \ldots Y_n$ depends only on

* The inherited attributes of the head non-terminal A.

* The attributes (synthesized or inherited) of symbols $Y_1 Y_2 \ldots Y_{k-1}$ located to the left of Y_k.

Analysis of Rule Sets:

- Rule 1 ($A \longrightarrow PQ$):

  * $P.i = A.i + 2$;

    Inherited attribute of P depends on A.i (Head attribute).

    ↳ valid

  * $Q.i = P.i + A.i$;

    Inherited attribute of Q depends on P.i (symbol to its left) and A.i (Head attribute) valid.

  * $A.s = P.s + Q.s$;

    Synthesized attribute of A depends on children P.s and Q.s

    ↳ valid.

• Rule 2 $(A \rightarrow XY)$:

* $X.i = A.i + Y.s$:

Inherited attribute of X depends on $Y.s$ which is a symbol to its right $\longrightarrow$ violation

* $Y.i = X.s + A.i$

Inherited attribute of Y depends on $X.s$ (symbol to its left) and $A.i$ (Head attribute) $\longrightarrow$ valid.

Thereby,

* Satisfies L → attribute definition: rule 1
* Violates L-attribute definition: rule 2 (due to the right to left dependency).

Q4: S — Attributed Syntax - Directed Definition

* $S \rightarrow E$; { S.code = E.code ; print (S.code) }
* $E \rightarrow E_1 + T$ { E.code = E_1.code || T.code || "+"; }
* $E \rightarrow E_1 - T$ { E.code = E_1.code || T.code || "-"; }
* $E \rightarrow T$ { E.code = T.code }
* $T \rightarrow T_1 * F$ { T.code = T_1.code || F.code || "*"; }
* $T \rightarrow T_1 / F$ { T.code = T_1.code || F.code || "/" }
* $T \rightarrow F$ { T.code = F.code; }
* $F \rightarrow (E)$ { F.code = E.code; }

* F → digit {F.code = digit.lexval; }

LR Parsing and Attribute Evaluation Trace :

(2+5*6)

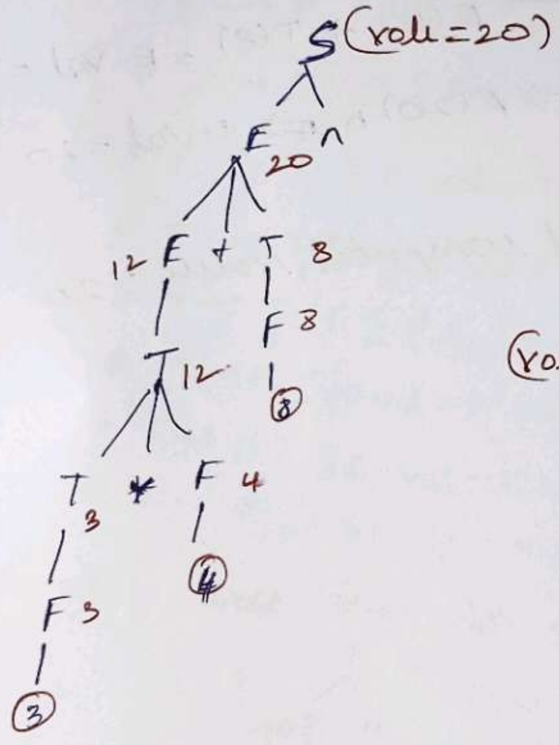
Step	Stack	Input	operation	computed Attribute
1	$	2+5*6 ; $	shift 2	–
2	$ digit 2	+5*6; $	Reduce F – digit	F.code = '2'
3	$F	+5*6; $	Reduce T –> F	T.code = '2'
4	$T	+ 5*6 ; $	Reduce E — T	E code = '2'
5	$ E	+5*6 ; $	shift +	–
6	$E+	+5*6 ; $	shift 5	F code = '5'
7	$E+digit	* 6 ; $	Reduce F –> digit	T.code = '5'
8	$E+F	* 6 ; $	Reduce T – F	
9	$E+T	* 6 ; $	shift *	–
10	$E+T*	6;$	shift 6	F.code = 6
11	$ E+T*dig(6)	; $	Reduce F – digit	T.code = "56*"
12	$E+T*F	; $	Reduce E → E+T	E.code = "256*+"
13	$E+T	; $	shift ;	–
14	$ E	; $	shift	
15	$ E ;	$	S –> E ;	S.code = "256*+"
16	$ S	$	Accept	Intermediate Postfix code generated

Postfix Intermediate code = 256*+

Q5 Syntax - Directed Definition (SDD);

* $S \rightarrow En$ { S.val = E.val }
* $E \rightarrow E_1 + T$ { E.val = E_1.val + T.val }
* $E \rightarrow T$ { E.val = T.val }
* $T \rightarrow T_1 * F$ { T.val = T_1.val * F.val }
* $T \rightarrow F$ { T.val = F.val }
* $F \rightarrow (E)$ { F.val = E.val }
* $F \rightarrow$ digit { F.val = digit.lexval }

Annotated parse tree for : 3*4+8n



(val = 20)

Top - Down Evaluation sequence & Final output.

1) Parse F ⟶ digit (3) => F.val = 3

2) Parse T ⟶ F => T.val = 3

3) Parse F ⟶ digit (4) => F.val = 4

4) compute T ⟶ T(3) * F(4) = T.val = 3×4 = 1

5) compute E ⟶ T(12) => E.val = 12

6) Parse F ⟶ digit (8) => F.val = 8

7) compute T ⟶ F(8) => T.val = 8

8) compute E ⟶ E(12) + T(8) = E.val = 12+8 = 20

9) compute S ⟶ E(20) n => S.val = 20

Final computed value : 20